Summary of Arrays and Functions A04

Ilex Travis

Roderick Crowder

ENGR-2304-3

9/8/2026

Arrays

Arrays in C Programming
- Collection of elements with the same data type stored in contiguous memory.
- Allows multiple values in one variable, accessed through an index starting at 0.
- Declared using data type, name, and size []; can also be initialized during declaration.
- Partially initialized arrays automatically set remaining elements to 0.
- Loops can efficiently access, print, or process elements.
- C supports multidimensional arrays, commonly used for matrices.
- Number of elements = total array size ÷ size of one element.

Key Considerations

Arrays in C have a fixed size, which must be defined at compile time.

All elements in an array must be of the same data type.

Arrays are passed to functions as pointers, and their size must be explicitly managed.

Arrays are a fundamental structure in C programming, enabling efficient data storage and manipulation for a wide range of applications.

Arrays: Syntax

In C programming, the general form of an array is data_type array_name[size]. The data type defines the elements (such as int, char, or double), the array name identifies it, and the size determines how many elements it can hold.

Local arrays without initialization contain indeterminate values, while static arrays without an initializer are zero-initialized. When initialized at declaration, values are assigned sequentially starting from index 0. Any remaining elements are filled with zeros, and if no size is given, C uses the number of initializers.

Array elements are accessed using the array name and index in square brackets. Since indexing starts at 0, valid indices range from 0 to size - 1. Overall, C array syntax is simple and direct, but you must pay close attention to the array size and how it is initialized.

Key Points

Fixed size: Cannot resize after creation; use dynamic allocation for flexibility.

Contiguous memory: Elements are stored next to each other for efficient access.

Indexing: Always use valid indices to avoid undefined behavior.

Pointer behavior: In many expressions, the array name is treated as a pointer to its first element.

Arrays: Formatting

In C programming, the general form of an array is data_type array_name[size]. The data type defines the elements (int, char, double), the name identifies the array, and the size determines how many elements it can hold.

Uninitialized local arrays contain indeterminate values, while static arrays are zero-initialized. When initialized, values are assigned sequentially starting from index 0. Any remaining elements are filled with zeros, and if no size is given, C uses the number of initializers.

Elements are accessed using the array name and index. Since indexing starts at 0, valid indices range from 0 to size - 1. Overall, C array syntax is simple and direct, but attention must be paid to the array size and initialization.

Key Points

Always initialize arrays to avoid undefined values.
Use correct indexing to prevent errors.
Be cautious with bounds to avoid memory corruption.
Multidimensional arrays require proper declaration and initialization.

Arrays : Benefits of Using Arrays

Arrays provide benefits such as one-dimensional or multidimensional storage, efficient constant-time indexing, and memory efficiency through contiguous storage. This also works well with CPU cache lines, improving performance. Arrays are versatile and can store different data types, including structures, while simplifying code and reducing copy-paste errors.

Common uses include storing grades, sensor readings, and game scores; creating matrices for math and graphics; buffering data streams or network packets; and supporting structures like stacks, queues, and dynamic programming tables.

Key Features

Contiguous memory layout - enables fast direct access via index calculation:
Address = Base_Address + (Index * sizeof(element_type)
Uniform data type - storage (int, float, chat, etc.)
Support for loops to process all elements efficiently
Pointer compatibility – Arrays can be passed to functions and used with pointers for dynamic manipulation.

To summarize arrays in C are fundamental and a very efficient way to store and process related data. Taking advantage of their contiguous memory layout, fast indexing, and hardware compatibility make them indispensable for performance-critical and embedded applications.

Arrays: Code Example

```
#include <stdio.h>
int main(void) {
    int grades[5];              /* local array – contains indeterminate values until initialized */
    int scores[5] = {95, 87, 92};   /* scores[0]=95, scores[1]=87, scores[2]=92, scores[3]=0, scores[4]=0 */
    char vowels[] = {'a', 'e', 'i', 'o', 'u'};  /* size is 5 */
    printf("First score: %d\n", scores[0]);         /* 95 */
    printf("Last score:  %d\n", scores[4]);         /* 0  */
    printf("Number of vowels: %zu\n", sizeof(vowels) / sizeof(vowels[0]));
    printf("All scores: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", scores[i]);
    }
    printf("\n");
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    printf("Matrix:\n");
    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 3; col++) {
            printf("%d ", matrix[row][col]);
        }
        printf("\n");
    }
    int *ptr = scores;          /* same as &scores[0] */
    printf("Via pointer: %d\n", *ptr);              /* 95 */
    printf("Next element: %d\n", *(ptr + 1));       /* 87 */
    return 0;
}
```

Functions

Functions allow you to break a complex program down into smaller, manageable pieces. A process known as modularization. Each function is designed to perform a specific, isolated task. Depending on its design, a function can accept input data and pass back output data. In c programming, a function is a self contained block of code that performs a specific isolated task.

Instead of writing the same logic repeatedly throughout your code, you group it into a named function that can be reused as many times as needed.

Functions: Syntax

The purpose of a C function is to allow the programmer to accomplish exactly this; that is, to move snippets of code outside of the main program and call them by an intuitive name. These functions can then be reused in any program where they are needed.

These parts goes in a specific place:
return-type function_name(param1, param2, ...){ }
Where param1, etc. are composed of two parts:
param1_type param1_name
Example:
int imax(int x, int y)
Return type : int
Name : imax
Parameter list : an integer x and an integer y

Components of a function

- Each function is composed of three parts:– Name
- Like variables it's up to you.
- Should choose clear, intuitive name– Return type
- Data type that the function will return to user.
- For imax function this is an int.– Parameter list
- List of datatypes that will be passed to the function.
- For imax functions there are two: int and int.– Author of function makes all of these decisions!

Functions: Formatting

For each problem on the previous slide, we should first set up the function before writing any code:
– What is the input type and number?
– What is the output (return val)?
– What is a good name for the function?
Once this is done, we can address go ahead and write the details of the function.

Just a few more things to know about functions:
– Prototype
– Dummy argument names
– Void argument lists and return types
– Calling a function without accepting return value
– "pass by value" vs. "pass by reference"

Prototyping
Though not always strictly necessary, you should template all functions.
Templating means the following:
– For each non-main function in your source file, you should include function headers just before main().
A "header" is just the return_type, name, and param list of the function, ie:
int imax(int a, int b)
I want you to understand that "dummy arguments" a and b are optional here they do not have to be used.
Int imax(int, int)
It is generally ok, when templating helps the compiler know what to expect when it encounters a function call for the first time.

Void arguments
• A function may have zero or one return values, and zero or more parameters. If you create a function with zero return values, you should use void:– void printerr(int errorcode);
• Likewise for zero parameters:– void printerr(void)
• Use void is optional, but it is a good practice since the compiler it will cause the compiler to do some extra checking.

Functions: Benefits of Using Functions

Calling Functions, so when functions that retrace not void, sometimes you will still choose to release the return argument by simply , what is known as calling the function, using the, no " = ".
– imax(a,b) is legal, though not obviously useful.
– c = imax(a,b) is more typical and useful

Benefits of functions
– when you are wanting to divide and conquer
- Helps with managing program development that could occur
– Another great thing is the software reusability, being able to use existing functions reuse them as parts and building blocks for new programs
- Another is the quality of abstraction - hide internal details (library functions)
– Avoid code repetition

When speaking about the formal process of calling a function it will go as follows:
return_value = func_name(arg1, arg2, …);
return_value
func_name
optional variable to store output, name of the function you wish to invoke. arg1, … type.
This becomes the data that you will send to passing to the function. Also known as, an argument list. You must always match the function parameter list in number and type. Very Important.

When using functions in c which will help a lot with situational coding. It is called dummy arguments. When you want to setup a function param names, often this is where the name is from.
For example, for imax:
– int imax(int a, int b)
– The variables a and b are the dummy arguments.
– You can choose to name these anything regardless of the specific names passed to the function at calling time.
– Whatever you choose to name them, that will be their name inside the function

Functions: Code Example

Imagine that many different times in a C program you have to find the maximum of two numbers:

```
if ( a > b ){
bigger = a;
}
else{
bigger = b;
}
```

• While this may not seem like a lot of code, it can get very ugly and make the program difficult to follow. It would be much nicer to say something like:

```
bigger = imax(a,b);
```

Write a function which takes no input and gives no output; it's only effect is to print an error message to the screen and abort.

• Write a function called power with the following interface:– double power(double x, int n) which evaluates x to the nth power.

• Write a function which returns the sum of an arbitrary-sized 1d array.

• Write a function that sorts an array

• Write a function to find the max value of an array

Using our previous imax example:

```
int maxval; /* declare an integer to
store the result */
int a = 8; b = 10; /* create some data */
maxval = imax(a,b); /* function call */
```

• Example says "call the function imax, passing to it the variables a and b, and store the result in the integer maxval."

Conclusion